

FIAP GRADUAÇÃO

DOMAIN DRIVEN DESIGN

Prof. Me. Thiago T. I. Yamamoto

#16 – API PADRÕES DE PROJETOS



TRAJETÓRIA



- ✓ Collections Framework
- ✓ Tratamento de Exceções
- ✓ JDBC
- ✓ JDBC 2
- ✓ Web Services Restul - API
- ✓ API Padrões de Projetos

#16 - AGENDA



- **Padrões de Projetos**
- **DTO**
- **Validações**
- **Mapeamento de Exceptions para Status HTTP**



PADRÕES DE PROJETOS

- **Padrões de projeto**, também conhecidos como Design Patterns em inglês, são **soluções reutilizáveis para problemas comuns** de design de software.
- Representam **melhores práticas que os desenvolvedores** de software podem aplicar para resolver desafios de design específicos ao criar aplicativos e sistemas.
- Os padrões de projeto ajudam a tornar o código mais eficiente, legível, flexível e facilmente mantido.

- **Um DTO (Data Transfer Object)** é um padrão de design utilizado para transferir dados entre diferentes camadas de uma aplicação, especialmente quando se deseja comunicar entre a camada de apresentação (API) e a camada de lógica de negócios.
- É uma "**representação simplificada**" dos objetos de domínio (**entidades**), proporcionando um encapsulamento dos dados a serem enviados ou recebidos, sem expor diretamente os detalhes internos do modelo.

- **Necessidade de controlar os dados enviados/recebidos:** Pode haver informações sensíveis ou internas que não devem ser expostas para o cliente.
- **Performance e simplificação:** O DTO permite retornar apenas os campos necessários, reduzindo o tamanho dos objetos transferidos.
- **Separação de responsabilidades:** Mantém a lógica de domínio separada da lógica de apresentação, facilitando a manutenção.

- **Crie uma classe para cada ação (Cadastrar, Atualizar e Pesquisar), com os dados que serão utilizados na API.**

```
package br.com.fiap.dto;

public class CadastroAlunoDto {

    private String nome;
    private String email;

    // Getters e Setters..

}
```

- Para melhorar a implementação do uso de DTOs, podemos utilizar a biblioteca `ModelMapper`, que facilita a conversão entre a entidade de domínio (Aluno) e o DTO
- O `ModelMapper` reduz a quantidade de código repetido para realizar essas conversões, tornando o processo mais simples e menos propenso a erros..

```
<dependency>  
  <groupId>org.modelmapper</groupId>  
  <artifactId>modelmapper</artifactId>  
  <version>3.1.1</version>  
</dependency>
```

- **Crie o bean**, para ser injetado nas classes **Resources**.

```
package br.com.fiap.bean;

import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.inject.Produces;
import org.modelmapper.ModelMapper;

public class ModelMapperProducer {

    @Produces
    @ApplicationScoped
    public ModelMapper modelMapper() {
        return new ModelMapper();
    }
}
```

- Na classe resource, crie um atributo do tipo **ModelMapper**.

```
@Path("/alunos")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class AlunoResource {

    @Inject
    private AlunoDao dao;

    @Inject
    private ModelMapper mapper;

    //...

}
```

```
@POST
public Response adicionar(CadastroAlunoDto dto, @Context UriInfo uriInfo)
    throws SQLException {

    Aluno aluno = mapper.map(dto, Aluno.class);
    dao.inserir(aluno);

    URI uri = uriInfo.getAbsolutePathBuilder()
        .path(String.valueOf(aluno.getId()))
        .build();

    return Response.created(uri)
        .entity(mapper.map(aluno, DetalhesAlunoDto.class))
        .build();
}
```

```
@PUT
@Path("/{id}")
public Response atualizar(@PathParam("id") int id, AtualizacaoAlunoDto dto)
                                                    throws SQLException {

    Aluno aluno = mapper.map(dto, Aluno.class);
    aluno.setId(id);
    dao.atualizar(aluno);
    return Response.ok().build();
}
```

```
@GET
@Path("/{id}")
public Response buscar(@PathParam("id") int id) throws SQLException {
    Aluno aluno = dao.buscar(id);
    return Response.ok(mapper.map(aluno, DetalhesAlunoDto.class)).build();
}
```

```
@GET  
public List<DetalhesAlunoDto> listar() throws SQLException {  
    return dao.listar()  
        .stream().map(aluno ->  
            mapper.map(aluno, DetalhesAlunoDto.class)).toList();  
}
```




VALIDAÇÕES

- É possível validar campos nulos, campos vazios, determinar o máximo e mínimo de caracteres de uma String, validar e-mails e também utilizar expressão regular na validação.

Anotação	Descrição
@NotNull	O campo não pode ser nulo.
@NotEmpty	O campo não pode ser nulo nem vazio (para strings).
@NotBlank	O campo não pode ser nulo nem conter apenas espaços (para strings).
@Size(min, max)	O tamanho do campo deve estar entre min e max.
@Email	O campo deve conter um endereço de e-mail válido.
@Pattern(regexp)	O campo deve corresponder à expressão regular regexp.

- Para números, é possível validar o valor máximo e mínimo, determinar se o valor deve ser positivo ou negativo.

Método	Descrição
@Min(value)	O valor do campo deve ser maior ou igual a value.
@Max(value)	O valor do campo deve ser menor ou igual a value.
@Positive	O valor do campo deve ser positivo.
@PositiveOrZero	O valor do campo deve ser positivo ou zero.
@Negative	O valor do campo deve ser negativo.
@NegativeOrZero	O valor do campo deve ser negativo ou zero.

- Podemos validar se a data deve estar no passado ou futuro. Também é possível determinar se um valor booleano deve ser falso ou true.

Método	Descrição
@Past	A data no campo deve ser no passado.
@Future	A data no campo deve ser no futuro.
@AssertTrue	O valor do campo deve ser verdadeiro (para booleanos).
@AssertFalse	O valor do campo deve ser falso (para booleanos).

- Vamos adicionar a dependência para realizar as validações:

```
<dependency>  
  <groupId>io.quarkus</groupId>  
  <artifactId>quarkus-hibernate-validator</artifactId>  
</dependency>
```

```
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public class CadastroAlunoDto {

    @NotBlank(message = "O nome é obrigatório")
    private String nome;

    @NotBlank(message = "O e-mail é obrigatório")
    @Email(message = "E-mail inválido")
    private String email;

    //get e set..

}
```

- A anotação **@Valid** é utilizada em métodos controladores, principalmente em métodos que recebem objetos de formulários da web ou solicitações REST como argumentos.
- Ela ativa a validação dos dados do objeto antes que o método do controlador seja executado.
- Deve ser colocado antes do objeto que queremos validar.

@POST

```
public Response adicionar(@Valid CadastroAlunoDto dto, @Context UriInfo uriInfo)
    throws SQLException {

    Aluno aluno = mapper.map(dto, Aluno.class);
    dao.inserir(aluno);

    URI uri = uriInfo.getAbsolutePathBuilder()
        .path(String.valueOf(aluno.getId()))
        .build();

    return Response.created(uri)
        .entity(mapper.map(aluno, DetalhesAlunoDto.class))
        .build();
}
```




EXCEPTIONS

- Lidar com exceções de forma estruturada é importante para garantir que as respostas retornadas aos clientes sejam claras, consistentes e com o status HTTP correto.
- Quarkus permite mapear exceções para respostas personalizadas usando a classe **ExceptionHandler**.
- O `ExceptionHandler` é uma interface que permite mapear exceções para respostas HTTP customizadas. A implementação de um `ExceptionHandler` traduz uma exceção Java em uma resposta HTTP apropriada, contendo status, headers e corpo.

- Crie uma classe para retornar o erro.

```
package br.com.fiap.exception;

public class ErrorResponseDto {
    private String erro;
    private String detalhe;

    public ErrorResponseDto(String erro, String detalhe) {
        this.erro = erro;
        this.detalhe = detalhe;
    }

    //getters e setters..

}
```

- Mapeamento das exceptions para um status code HTTP.

```
@Provider
public class ApiExceptionHandler implements ExceptionMapper<Exception> {

    @Override
    public Response toResponse(Exception exception) {
        if (exception instanceof EntidadeNaoEncontradaException){
            return Response.status(Response.Status.NOT_FOUND)
                .entity(new ErrorResponseDto("Entidade não encontrada",exception.getMessage())).build();
        } else if (exception instanceof NotFoundException){
            return Response.status(Response.Status.NOT_FOUND).build();
        } else if (exception instanceof SQLException){
            return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
                .entity(new ErrorResponseDto("SQL Exception",exception.getMessage())).build();
        }
        return Response.status(Response.Status.INTERNAL_SERVER_ERROR).build();
    }
}
```

- Mapeamento de exceptions para validações de dados.

```
@Provider
public class ApiValidationMapper implements ExceptionMapper<ConstraintViolationException> {
    @Override
    public Response toResponse(ConstraintViolationException exception) {
        StringBuilder errors = new StringBuilder();
        exception.getConstraintViolations().forEach(violation ->
            errors.append(violation.getMessage()).append("; ")
        );
        return Response.status(Response.Status.BAD_REQUEST)
            .entity(new ErrorResponseDto("Dados inválidos", errors.toString()))
            .build();
    }
}
```

Copyright © 2020 - 2025
Prof. Thiago T. I. Yamamoto

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).

*“O que a pessoa pensa a respeito de si mesmo indica o seu destino”.
(Henry David Thoreau, adaptado)*